Very Brief Machine Learning

Overview

- You may have heard of machine learning/AI/ChatGPT…
- Full subject way too big to tackle here
- We'll restrict ourselves to one example - classifying handwritten digits.
- Dataset we'll use is modified NIST (MNIST). Handwritten digits by high schoolers + NIST employees, with known answers.
- Much of discussion motivated by http://neuralnetworksanddeeplearning.com/ by Michael Nielsen.

Supervised Learning

- We have a bunch of input images, and we know what the answer should be.
- This is *supervised learning*. c.f. unsupervised learning, where nobody gave you answers.
- We want function that outputs correct digit given input image.
- We want this to work on data *we have not yet seen*. This is not the same as explaining data we already have.

Universal Functions

- Previously, we've had an idea what data should look like. e.g. polynomial - we a) know it's a polynomial, and b) can predict data by evaluating polynomial given coeffs.

- Challenge has been finding m given A such that d=A(m)

- Now we have a bunch of input parameters (the images), and the output data (what digit we have). Challenge is to find A.

- *Exceedingly* hard to write down A a priori. What is the function for "Is this an 8? Is this a cat?"?

- One solution - use a universal function for A that can model *any* function given parameters of A.

Neutrons/Perceptrons

- Nature has figured out one way to do this. Neurons get input (either 1 or 0) from other neurons. If sum of inputs greater than a threshold, output 1, otherwise 0.

- Neurons can be more sensitive to some neurons than others. We assign a weight to each connection.

- If we have an input layer of neurons (subscript i) connected to another output layer (subscript j), then we want to know if $\sum W_{ij}x_j > t_i$ where W_{ij} is the weight of the j^{th} input neuron feeding into i^{th} output neuron. x_j is signal from j^{th} neuron, and t_i is threshold for i^{th} neuron to fire.

Bias and Heaviside

- Move threshold to left-hand side and rename bias $b_i=-t_i$.
- Heaviside function $H(x)=1$ if $x>0$, 0 if $x<0$.
- Output of second layer neuron becomes $H(W_{ij}x_j+b_i)$
- We can stack layers to get more complicated behavior (the "deep" in deep learning). Interior layers are called "hidden"
- Binary output neurons are called perceptrons.
- As written, every neuron can speak to every neuron in next layer. Your brain doesn't need this (W is mostly zeros).

Two Examples

- Do we go to the cheese festival?

- Can we make a NAND gate? If yes, we can build arbitrary computer out of perceptrons (since NAND universal)

Sigmoid and Universal

- We're going to fit our parameters ("train the network") by taking gradients. H is bad for training because gradient is either zero or infinity.

- We'll replace H by something that goes smoothly from 0 to 1 around x=0. These are called sigmoids ('cause roughly s-shaped).

- Output becomes S(Wx+b), where S is called activation function. One simple is logistic - exp(x)/(1+exp(x)).

- Cybenko (1989) proved that 2-layer network can approximate any function under some weak constraints on S, as long as S is non-linear.

Cost Functions

- Given a bunch of training data (input to first layer of neurons) and target values (desired output of last layer), and having chosen S, we want to find the “best” W’s and b’s

- We need a mathematical definition of “best”. We’ll use familiar squared errors: $\sum(\text{output-target})^2$. Other choices possible; general term is “Cost Function”. When you hear that, think chi^2.

- In familiar language, we want to minimize $(d-A(m))^T N^{-1}(d-A(m))$, except (many!) m’s and d’s are given, and we want to find A.

Problem Setup

- You might think output should be what digit we have. This is bad.
- What happens if network isn't sure if you have a 3 or 8? Output of 5.5 would not make much sense.
- Instead, have 10 outputs. First output: "is this a 1?", second output "Is this a 2?", with output range 0 (this is not a 2) to 1 (this is definitely a 2).
- Position of largest output is best guess for value.

Backpropagation 1

- We're ready to calculate gradients. For single data, we want dC/dp for parameter p, then we sum over data. Set N=I

- Look at *last* layer: dC/dp= d/dp((y-S(Wx+b))T(y-S(Wx+b)))

- Let r=y-S(), this is difference between output and truth.

- $dC/db=-2r^T d(S)/db$, d(S)/db=S'

- $dC/dW=-2r^T d(S)/dW$, d(S)/dW=S'()x

- Also need $dC/dx = -2r^T d(S)/dx$, d(S)/dx=S'()W

Backpropagation 2

- How does cost depend on next-to-last layer? Given input x, changing W,b changes output, which is input to the final layer.

- We already know how cost depends on input to final layer, so we can use chain rule to find how cost depends on previous layer.

- Repeat through network, and we have gradient of cost w.r.t each parameter in the network. This is called backpropagation (and was big part of Hinton's Nobel prize).

- Look at net_example.py. It calculates gradients, with multiple input data chunks allowed.

SGD

- Now that we have gradients, we want to minimize cost.
- We have a ton of parameters - tens of thousands for MNIST, many billions for the models you've heard of.
- Would be lovely to calculate full curvature, but we can't afford that. Instead, will go downhill.
- "Isn't that bad?" Yeah, but we're kind of stuck. Turns out looping over chunks of data and going a bit downhill works reasonably well.
- This is called stochastic gradient descent - we randomly pick chunks ("minibatches") of data, move a little downhill, and repeat.
- "How far downhill should we go?" Great question!